

Day 17

System Verilog: Functional Coverage

Cover points for FIFO:

```
module fifo_with_assertions #(
    parameter DEPTH = 8,
    parameter WIDTH = 8
)(
    input logic      clk, rst,
    input logic      wr_en, rd_en,
    input logic [WIDTH-1:0] din,
    output logic [WIDTH-1:0] dout,
    output logic      full, empty
);
    reg [WIDTH-1:0] mem [0:DEPTH-1];
    reg [$clog2(DEPTH):0] wr_ptr, rd_ptr, count;
    // write
    always @(posedge clk) begin
        if(rst) wr_ptr <= 0;
        else if(wr_en && !full) begin
            mem[wr_ptr] <= din;
            wr_ptr <= (wr_ptr == DEPTH-1) ? 0 : wr_ptr + 1;
        end
    end
    //read
    always @(posedge clk) begin
        if(rst) rd_ptr <= 0;
        else if(rd_en && !empty) begin
            dout <= mem[rd_ptr];
            rd_ptr <= (rd_ptr == DEPTH-1) ? 0 : rd_ptr + 1;
        end
    end
    always @(posedge clk) begin
        if(rst) count <= 0;
        else begin
            case({wr_en & !full, rd_en & !empty})
                2'b10: count <= count+1;
                2'b01: count <= count-1;
                2'b11: count <= count;
                default: count <= count;
            endcase
        end
    end

    assign full = (count == DEPTH);
    assign empty = (count == 0);

    // — Assertion 1: never write when full —————
    property p_no_write_when_full;
        @(posedge clk) disable iff(rst)
            full |-> !wr_en;
    endproperty
    assert property(p_no_write_when_full)
    else $error("Write attempted when FIFO full");
```

```

// — Assertion 2: never read when empty —————
property p_no_read_when_empty;
    @(posedge clk) disable iff(rst)
    empty |-> !rd_en;
endproperty
assert property(p_no_read_when_empty)
else $error("Read attempted when FIFO empty");

// — Assertion 3: full and empty never both true —————
property p_not_full_and_empty;
    @(posedge clk) disable iff(rst)
    !(full && empty);
endproperty
assert property(p_not_full_and_empty)
$info("Success: FIFO is not full and empty simulataneously");
else $error("Fail: FIFO full AND empty simultaneously ie impossible");

// — Assertion 4: after reset both flags correct —————
property p_reset_state;
    @(posedge clk)
    $rose(rst) |=> (empty == 1 && full == 0);
endproperty
assert property(p_reset_state)
$info("Success: Reset");
else $error("Fail: Wrong state after reset");

// — Assertion 5: full deasserts after a read —————
property p_full_clears_on_read;
    @(posedge clk) disable iff(rst)
    (full && rd_en) |=> !full;
endproperty
assert property(p_full_clears_on_read)
$info("Success: Full flag cleared the read");
else $error("Fail: Full flag did not clear after read");

// — Assertion 6: empty deasserts after a write —————
property p_empty_clears_on_write;
    @(posedge clk) disable iff(rst)
    (empty && wr_en) |=> !empty;
endproperty
assert property(p_empty_clears_on_write)
$info("Success: Empty flag cleared after write");
else $error("Fail: Empty flag did not clear after write");

```

endmodule

Testbench with coverage points:

```

module tb_fifo_coverage;
    parameter DEPTH = 8;
    parameter WIDTH = 8;

    logic      clk, rst;
    logic      wr_en, rd_en;
    logic [WIDTH-1:0] din, dout;
    logic      full, empty;

```

```

logic [$clog2(DEPTH):0] count; // expose count for coverage

fifo_with_assertions #(DEPTH, WIDTH) dut(
    .clk(clk), .rst(rst),
    .wr_en(wr_en), .rd_en(rd_en),
    .din(din), .dout(dout),
    .full(full), .empty(empty)
);

initial begin clk = 0; forever #5 clk = ~clk; end

// — Define the covergroup —————
covergroup fifo_cg @(posedge clk);

    // Coverpoint 1: FIFO occupancy levels
    // Did we test empty, quarter-full, half-full, and full?
    cp_count: coverpoint dut.count {
        bins empty      = {0};
        bins quarter_full = {[1:2]};
        bins half_full   = {[3:5]};
        bins nearly_full = {[6:7]};
        bins full        = {8};
    }

    // Coverpoint 2: write enable states
    cp_wr: coverpoint wr_en {
        bins wr_active   = {1};
        bins wr_inactive = {0};
    }

    // Coverpoint 3: read enable states
    cp_rd: coverpoint rd_en {
        bins rd_active   = {1};
        bins rd_inactive = {0};
    }

    // Coverpoint 4: flag states
    cp_flags: coverpoint {full, empty} {
        bins both_zero = {2'b00}; // normal operation
        bins empty_only = {2'b01}; // FIFO empty
        bins full_only = {2'b10}; // FIFO full
        // 2'b11 is illegal — full AND empty simultaneously
        illegal_bins both_set = {2'b11};
    }

    // Coverpoint 5: data values — corner cases
    cp_data: coverpoint din {
        bins zeros = {8'h00};
        bins ones = {8'hFF};
        bins walking1 = {8'h01, 8'h02, 8'h04, 8'h08,
                        8'h10, 8'h20, 8'h40, 8'h80};
        bins others = default;
    }

    // Cross coverage — combinations of coverpoints
    // Did we test writing when full? Reading when empty?
    cross_wr_flags: cross cp_wr, cp_flags, cp_rd { // Added cp_rd here so you can use it below

```

```

// binsof(CoverpointName) intersect {ValueOrBinName}
bins write_when_full = binsof(cp_wr) intersect {1} && binsof(cp_flags.full_only);

bins read_when_empty = binsof(cp_rd) intersect {1} && binsof(cp_flags.empty_only);
}

endgroup

// Instantiate the covergroup
fifo_cg cg = new();

// — Transaction class —————
class fifo_txn;
  rand logic [7:0] data;
  rand logic      wr_en;
  rand logic      rd_en;

  constraint rw_excl { !(wr_en && rd_en); }
  constraint not_idle { wr_en || rd_en; }
endclass

// — Main test —————
initial begin
  automatic fifo_txn txn = new();

  cg.start();

  rst = 1; wr_en = 0; rd_en = 0; din = 0;
  #20; rst = 0;

  // Run random transactions
  repeat(200) begin
    assert(txn.randomize());
    @(negedge clk);
    wr_en = txn.wr_en;
    rd_en = txn.rd_en;
    din = txn.data;
    @(posedge clk); #1;
    wr_en = 0; rd_en = 0;
  end
  cg.stop();
  // Print coverage report
  $display("\n=== Coverage Report ===");
  $display("Overall coverage: %0.2f%%", cg.get_coverage());
  $display("cp_count coverage: %0.2f%%", cg.cp_count.get_coverage());
  $display("cp_flags coverage: %0.2f%%", cg.cp_flags.get_coverage());
  $display("cp_data coverage: %0.2f%%", cg.cp_data.get_coverage());

  #20; $finish;
end
endmodule

```

Output:

xrun: 25.03-s001: (c) Copyright 1995-2025 Cadence Design Systems, Inc.
 Top level design units:

tb_fifo_coverage

```
xcelium> source /xcelium25.03/tools/xcelium/files/xmsimrc
xcelium> run
xmsim: *N,ASRTST (./design.sv,76): (time 15 NS) Assertion tb_fifo_coverage.dut.p_reset_state has passed
Success: Reset
xmsim: *N,ASRTST (./design.sv,67): (time 25 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,94): (time 35 NS) Assertion tb_fifo_coverage.dut.p_empty_clears_on_write has
passed
Success: Empty flag cleared after write
xmsim: *N,ASRTST (./design.sv,67): (time 35 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 45 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,94): (time 55 NS) Assertion tb_fifo_coverage.dut.p_empty_clears_on_write has
passed
Success: Empty flag cleared after write
xmsim: *N,ASRTST (./design.sv,67): (time 55 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 65 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 75 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 85 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *E,ASRTST (./design.sv,59): (time 85 NS) Assertion tb_fifo_coverage.dut.p_no_read_when_empty has failed
Read attempted when FIFO empty
xmsim: *N,ASRTST (./design.sv,67): (time 95 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,94): (time 105 NS) Assertion tb_fifo_coverage.dut.p_empty_clears_on_write has
passed
Success: Empty flag cleared after write
xmsim: *N,ASRTST (./design.sv,67): (time 105 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 115 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 125 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 135 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 145 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 155 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 165 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 175 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 185 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 195 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 205 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 215 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
```

[illegible]

[illegible]

[illegible]

passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,85): (time 1255 NS) Assertion tb_fifo_coverage.dut.p_full_clears_on_read has passed

Success: Full flag cleared the read

xmsim: *N,ASRTST (./design.sv,67): (time 1255 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1265 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1275 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1285 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1295 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1305 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1315 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1325 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *E,ASRTST (./design.sv,51): (time 1325 NS) Assertion tb_fifo_coverage.dut.p_no_write_when_full has failed
Write attempted when FIFO full

xmsim: *N,ASRTST (./design.sv,67): (time 1335 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,85): (time 1345 NS) Assertion tb_fifo_coverage.dut.p_full_clears_on_read has passed
Success: Full flag cleared the read

xmsim: *N,ASRTST (./design.sv,67): (time 1345 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1355 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1365 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1375 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,85): (time 1385 NS) Assertion tb_fifo_coverage.dut.p_full_clears_on_read has passed
Success: Full flag cleared the read

xmsim: *N,ASRTST (./design.sv,67): (time 1385 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1395 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,85): (time 1405 NS) Assertion tb_fifo_coverage.dut.p_full_clears_on_read has passed
Success: Full flag cleared the read

xmsim: *N,ASRTST (./design.sv,67): (time 1405 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1415 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1425 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1435 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *E,ASRTST (./design.sv,51): (time 1435 NS) Assertion tb_fifo_coverage.dut.p_no_write_when_full has failed
Write attempted when FIFO full

xmsim: *N,ASRTST (./design.sv,67): (time 1445 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *E,ASRTST (./design.sv,51): (time 1445 NS) Assertion tb_fifo_coverage.dut.p_no_write_when_full has failed
Write attempted when FIFO full

xmsim: *N,ASRTST (./design.sv,67): (time 1455 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *E,ASRTST (./design.sv,51): (time 1455 NS) Assertion tb_fifo_coverage.dut.p_no_write_when_full has failed
Write attempted when FIFO full

xmsim: *N,ASRTST (./design.sv,67): (time 1465 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *E,ASRTST (./design.sv,51): (time 1465 NS) Assertion tb_fifo_coverage.dut.p_no_write_when_full has failed
Write attempted when FIFO full

xmsim: *N,ASRTST (./design.sv,67): (time 1475 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,85): (time 1485 NS) Assertion tb_fifo_coverage.dut.p_full_clears_on_read has passed
Success: Full flag cleared the read

xmsim: *N,ASRTST (./design.sv,67): (time 1485 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1495 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *E,ASRTST (./design.sv,51): (time 1495 NS) Assertion tb_fifo_coverage.dut.p_no_write_when_full has failed
Write attempted when FIFO full

xmsim: *N,ASRTST (./design.sv,67): (time 1505 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,85): (time 1515 NS) Assertion tb_fifo_coverage.dut.p_full_clears_on_read has passed
Success: Full flag cleared the read

xmsim: *N,ASRTST (./design.sv,67): (time 1515 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *N,ASRTST (./design.sv,67): (time 1525 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

xmsim: *E,ASRTST (./design.sv,51): (time 1525 NS) Assertion tb_fifo_coverage.dut.p_no_write_when_full has failed
Write attempted when FIFO full

xmsim: *N,ASRTST (./design.sv,67): (time 1535 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously
xmsim: *E,ASRTST (./design.sv,51): (time 1535 NS) Assertion tb_fifo_coverage.dut.p_no_write_when_full has failed
Write attempted when FIFO full
xmsim: *N,ASRTST (./design.sv,67): (time 1545 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *E,ASRTST (./design.sv,51): (time 1545 NS) Assertion tb_fifo_coverage.dut.p_no_write_when_full has failed
Write attempted when FIFO full
xmsim: *N,ASRTST (./design.sv,67): (time 1555 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,85): (time 1565 NS) Assertion tb_fifo_coverage.dut.p_full_clears_on_read has passed
Success: Full flag cleared the read
xmsim: *N,ASRTST (./design.sv,67): (time 1565 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1575 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1585 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1595 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,85): (time 1605 NS) Assertion tb_fifo_coverage.dut.p_full_clears_on_read has passed
Success: Full flag cleared the read
xmsim: *N,ASRTST (./design.sv,67): (time 1605 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1615 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *E,ASRTST (./design.sv,51): (time 1615 NS) Assertion tb_fifo_coverage.dut.p_no_write_when_full has failed
Write attempted when FIFO full
xmsim: *N,ASRTST (./design.sv,67): (time 1625 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,85): (time 1635 NS) Assertion tb_fifo_coverage.dut.p_full_clears_on_read has passed
Success: Full flag cleared the read
xmsim: *N,ASRTST (./design.sv,67): (time 1635 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1645 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1655 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1665 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1675 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1685 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed

Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1695 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1705 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *E,ASRTST (./design.sv,51): (time 1705 NS) Assertion tb_fifo_coverage.dut.p_no_write_when_full has failed
Write attempted when FIFO full
xmsim: *N,ASRTST (./design.sv,67): (time 1715 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,85): (time 1725 NS) Assertion tb_fifo_coverage.dut.p_full_clears_on_read has passed
Success: Full flag cleared the read
xmsim: *N,ASRTST (./design.sv,67): (time 1725 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1735 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1745 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1755 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1765 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1775 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1785 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1795 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1805 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1815 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1825 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1835 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1845 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1855 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1865 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has

passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1875 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,85): (time 1885 NS) Assertion tb_fifo_coverage.dut.p_full_clears_on_read has passed
Success: Full flag cleared the read
xmsim: *N,ASRTST (./design.sv,67): (time 1885 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1895 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *E,ASRTST (./design.sv,51): (time 1895 NS) Assertion tb_fifo_coverage.dut.p_no_write_when_full has failed
Write attempted when FIFO full
xmsim: *N,ASRTST (./design.sv,67): (time 1905 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,85): (time 1915 NS) Assertion tb_fifo_coverage.dut.p_full_clears_on_read has passed
Success: Full flag cleared the read
xmsim: *N,ASRTST (./design.sv,67): (time 1915 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1925 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1935 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1945 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1955 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1965 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1975 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1985 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 1995 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 2005 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 2015 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has passed
Success: FIFO is not full and empty simulataneously

=== Coverage Report ===

Overall coverage: 89.44%

cp_count coverage: 100.00%

```

cp_flags coverage: 100.00%
cp_data coverage: 66.67%
xmsim: *N,ASRTST (./design.sv,67): (time 2025 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has
passed
Success: FIFO is not full and empty simulataneously
xmsim: *N,ASRTST (./design.sv,67): (time 2035 NS) Assertion tb_fifo_coverage.dut.p_not_full_and_empty has
passed
Success: FIFO is not full and empty simulataneously
Simulation complete via $finish(1) at time 2036 NS + 0

```

The coverage report shows the overall coverage here which is 89 % , only the cover point of data needs to be covered.

To increase the coverage percentage, I am explicitly adding few more testcases here

```

repeat(200) begin
    assert(txn.randomize());
    @(negedge clk);
    wr_en = txn.wr_en & !full;
    rd_en = txn.rd_en & !empty;

    din = txn.data;
    @(posedge clk); #1;
    wr_en = 0; rd_en = 0;
    wr_en = 1; din = 8'h00; // force write zero
    @(negedge clk); wr_en = 0;

    @(negedge clk);
    wr_en = 1; din = 8'hFF;
    @(negedge clk); wr_en = 0;

end

```

Output:

```
xcelium> run
```

```

=== Coverage Report ===
Overall coverage: 96.67%
cp_count coverage: 100.00%
cp_flags coverage: 100.00%
cp_data coverage: 100.00%

```

We can see that the coverage has increased to 96.67%

Coverage-driven loop: running until covered, here the coverage will be taking place till it reaches 95% coverage point or 10,000 iterations

```

module tb_fifo_coverage;
    parameter DEPTH = 8;
    parameter WIDTH = 8;

    logic clk, rst;
    logic wr_en, rd_en;
    logic [WIDTH-1:0] din, dout;
    logic full, empty;

    // Instance of the FIFO
    fifo #(DEPTH, WIDTH) dut (
        .clk(clk),

```



```

.rst(rst),
.wr_en(wr_en),
.rd_en(rd_en),
.din(din),
.dout(dout),
.full(full),
.empty(empty)
);

// Clock Generation
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// — Define the covergroup —————
covergroup fifo_cg @(posedge clk);
    // Coverpoint 1: FIFO occupancy levels
    cp_count: coverpoint dut.count {
        bins empty = {0};
        bins quarter_full = {[1:2]};
        bins half_full = {[3:5]};
        bins nearly_full = {[6:7]};
        bins full = {8};
    }

    // Coverpoint 2: write enable states
    cp_wr: coverpoint wr_en {
        bins wr_active = {1};
        bins wr_inactive = {0};
    }

    // Coverpoint 3: read enable states
    cp_rd: coverpoint rd_en {
        bins rd_active = {1};
        bins rd_inactive = {0};
    }

    // Coverpoint 4: flag states
    cp_flags: coverpoint {full, empty} {
        bins both_zero = {2'b00};
        bins empty_only = {2'b01};
        bins full_only = {2'b10};
        illegal_bins both_set = {2'b11};
    }

    // Coverpoint 5: data values
    cp_data: coverpoint din {
        bins zeros = {8'h00};
        bins ones = {8'hFF};
        bins walking1 = {8'h01, 8'h02, 8'h04, 8'h08, 8'h10, 8'h20, 8'h40, 8'h80};
        bins others = default;
    }

    // Cross coverage
    cross_wr_flags: cross cp_wr, cp_flags, cp_rd {
        bins write_when_full = binsof(cp_wr) intersect {1} && binsof(cp_flags.full_only);
    }

```

```

        bins read_when_empty = binsof(cp_rd) intersect {1} && binsof(cp_flags.empty_only);
    }
endgroup

// Instantiate the covergroup
fifo_cg cg = new();

// — Transaction class —————
class fifo_txn;
    rand logic [7:0] data;
    rand logic wr_en;
    rand logic rd_en;
    constraint rw_excl { !(wr_en && rd_en); }
    constraint not_idle { wr_en || rd_en; }
endclass

// — Main Simulation Block —————
initial begin
    // Variables MUST be declared at the start of the block
    automatic fifo_txn txn = new();
    int unsigned iterations = 0;
    real coverage = 0.0;

    // Initialize signals
    rst = 1;
    wr_en = 0;
    rd_en = 0;
    din = 0;

    #20;
    rst = 0;
    cg.start();

    $display("Starting simulation...");

    // Run until 95% coverage or 10,000 iterations
    while(coverage < 95.0 && iterations < 10000) begin
        if(!txn.randomize()) $fatal("Randomization failed");

        @(negedge clk);
        // Drive signals (with simple overflow/underflow protection)
        wr_en = txn.wr_en;
        rd_en = txn.rd_en;
        din = txn.data;

        @(posedge clk);
        #1; // Wait for sampling to occur

        coverage = cg.get_coverage();
        iterations = iterations + 1;

        if(iterations % 500 == 0)
            $display("Iteration %0d: Current Coverage = %0.2f%%", iterations, coverage);
    end

    cg.stop();

```

```
// — Final Coverage Report —————
$display("\n=====");
$display("  FINAL COVERAGE REPORT");
$display("=====");
$display("Overall coverage: %0.2f%%", cg.get_coverage());
$display("Occupancy (count): %0.2f%%", cg.cp_count.get_coverage());
$display("Flags coverage:  %0.2f%%", cg.cp_flags.get_coverage());
$display("Data coverage:  %0.2f%%", cg.cp_data.get_coverage());
$display("Total Iterations: %0d", iterations);
$display("=====\\n");

if(coverage >= 95.0)
  $display("RESULT: PASS: Coverage goal reached!");
else
  $display("RESULT: FAIL: Goal not met within iteration limit.");

#20;
$finish;
end

endmodule
```

Output:

```
xcelium> run
Starting simulation...
```

```
=====
  FINAL COVERAGE REPORT
=====
Overall coverage: 95.00%
Occupancy (count): 100.00%
Flags coverage:  100.00%
Data coverage:   100.00%
Total Iterations: 57
=====
```

```
RESULT: PASS: Coverage goal reached!
```